Financial Fraud Detection System

End-to-End Data Analysis, Machine Learning & Real-Time Monitoring Project

1. Objective

This project analyzes financial transaction data to identify fraud patterns, trains and compares multiple machine learning models, and implements a real-time monitoring system that scores incoming transactions and raises alerts for high-risk cases. The system is visualized through an interactive Streamlit dashboard.

2. Dataset

The dataset consists of 5,000 financial transactions with 14 original attributes, including transaction amount, merchant category, payment method, device type, location, international flag, account age, and a suspicious-keyword indicator. The overall fraud rate is 9.64% (482 fraudulent transactions), making this an imbalanced classification problem.

3. System Architecture

Raw CSV → SQLite Database (ETL) → Feature Engineering → Model Training (Logistic Regression / Random Forest / Isolation Forest) → Best Model Saved → Simulated Real-Time Stream → Alert System → Streamlit Dashboard

4. Data Cleaning & Exploratory Data Analysis

The dataset required no missing-value or duplicate handling. The transaction date field was converted to a proper datetime format. Exploratory analysis revealed the following fraud-rate patterns across key features:

Feature	Fraud Rate Impact
Suspicious Keyword present	47.3% vs 7.6% (about 6x higher)
International transaction	37% vs 7% (about 5x higher)
Night hours (12 AM - 5 AM)	20-38% vs 5-8% during the day
Location, merchant category, day of week	Weak impact
Transaction amount, account age, device type	Negligible impact

5. Feature Engineering

Two new binary features were engineered based on EDA findings: Suspicious_Keyword_Flag and Is_Night. Categorical columns (Merchant Category, Device Type, Payment Method) were one-hot encoded, resulting in a final set of 20 model-ready features.

6. ETL - Database Layer

The cleaned and encoded dataset was loaded into a SQLite database (fraud_detection.db) rather than being kept as a static CSV file. This mirrors how real-world systems store and retrieve transactional data, and allows the downstream streaming and dashboard components to query a live data source.

7. Model Training & Comparison

Three models were trained and evaluated using an 80-20 stratified train-test split, which preserves the original fraud ratio in both sets:

Model	Type	Precision (Fraud)	Recall (Fraud)	ROC-AUC
Logistic Regression	Supervised	27%	100%	0.885
Random Forest	Supervised	34%	32%	0.884
Isolation Forest	Unsupervised	36%	36%	N/A

Selected Model: Logistic Regression. In fraud detection, missing an actual fraud case (a false negative) is typically more costly than raising a false alarm. Logistic Regression achieved 100% recall, meaning no fraud cases were missed on the test set, and it remains the most interpretable of the three models.

8. Feature Importance Validation

The trained model's coefficients independently confirmed the top three EDA findings: Suspicious_Keyword_Flag (coefficient 4.52), Is_International (4.28), and Is_Night (4.14) emerged as the strongest predictors of fraud. All remaining features had coefficients close to zero, indicating little to no predictive contribution. This agreement between manual EDA and independent model learning strengthens confidence in the findings.

9. Real-Time Simulation

A Python-based streaming simulator processes transactions sequentially with a one-second interval between each, emulating the behavior of a live transaction feed without requiring heavier infrastructure such as Kafka or Spark. Each transaction is scored by the saved model in real time.

10. Alert System

Transactions whose predicted fraud probability exceeds a configurable threshold are automatically flagged. Each alert is timestamped and written to a persistent log file, creating an auditable record of flagged activity.

11. Interactive Dashboard

A Streamlit-based dashboard provides live visibility into the system, including summary metrics (total transactions, fraud count, fraud rate), location-wise and hour-wise fraud rate charts, a recent-alerts log view, and a filterable transaction table.

12. Precision-Recall Trade-off

A precision-recall curve was analyzed across decision thresholds. At the default threshold of 0.5, recall is maximized (100%) at the cost of precision (27%). Raising the threshold toward 0.75-0.9 improves precision at the expense of recall, giving the business a tunable lever depending on whether minimizing missed fraud or minimizing false alarms is the priority.

13. Conclusion

The system successfully achieves high-recall fraud detection using a small set of interpretable features, with Suspicious Keyword, International status, and Night-time transactions identified as the dominant fraud signals. The addition of a database layer, real-time simulation, alerting, and a live dashboard extends the project beyond a static analysis notebook into a working, deployable prototype.

14. Limitations & Future Scope

- Replace the simulated stream with real Kafka/Spark infrastructure for production deployment.
- Improve precision using advanced models such as XGBoost combined with threshold tuning.
- Integrate real email/SMS-based alerting instead of log-file-based alerts.
- Add a periodic model-retraining pipeline to handle concept drift as fraud patterns evolve.